

Министерство науки и высшего образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №7

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в глубину»

Выполнили:

студенты группы 24ВВВЗ

Задоркин Тимофей

Комиссаров Андрей

Приняли:

Юрова О.В.

Деев М.В.

Пенза 2025

Цель работы – ознакомиться с алгоритмом обхода графа в глубину и освоить его практическую реализацию.

Общие сведения.

Обход графа – одна из наиболее распространенных операций с графами. Задачей обхода является прохождение всех вершин в графе. Обходы применяются для поиска информации, хранящейся в узлах графа, нахождения связей между вершинами или группами вершин и т.д.

Одним из способов обхода графов является поиск в глубину. Идея такого обхода состоит в том, чтобы начав обход из какой-либо вершины всегда переходить по первой встречающейся в процессе обхода связи в следующую вершину, пока существует такая возможность. Как только в процессе обхода исчерпаются возможности прохода, необходимо вернуться на один шаг назад и найти следующий вариант продвижения. Таким образом, итерационно выполняя описанные операции, будут пройдены все доступные для прохождения вершины. Чтобы не заходить повторно в уже пройденные вершины, необходимо их пометить как пройденные.

Таким образом, можно предложить следующую рекурсивную реализацию алгоритма обхода в глубину.

Вход: G – матрица смежности графа.

Выход: номера вершин в порядке их прохождения на экране.

Алгоритм ПОГ

1.1. для всех i положим $NUM[i] = False$ пометим как "не посещенную";

1.2. **ПОКА** существует "новая" вершина v

1.3. **ВЫПОЛНЯТЬ** DFS (v).

Алгоритм DFS(v):

2.1. пометить v как "посещенную" $NUM[v] = \text{True}$;

2.2. вывести на экран v ;

2.3. **ДЛЯ** $i = 1$ **ДО** size_G **ВЫПОЛНЯТЬ**

2.4. **ЕСЛИ** $G(v,i) = 1$ **И** $NUM[i] = \text{False}$

2.5. **ТО**

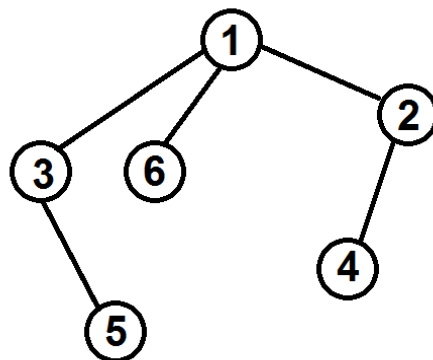
2.6. {

2.7. DFS(i);

2.8. }

Реализация состоит из подготовительной части, в которой все вершины помечаются как не помеченные (п.1.1) и осуществляется запуск процедуры обхода для вершин графа (п.1.2, 1.3). И непосредственно процедуры обхода, которая помечает текущую (т.е. ту, в которой на текущей итерации находится алгоритм) вершину как посещенную (п. 2.1). Затем выводит номер текущей вершины на экран (п.2.2) и в цикле просматривает v -ю строку матрицы смежности графа $G(v,i)$. Как только алгоритм встречает смежную с v не посещенную вершину (п.2.4), то для этой вершины вызывается процедура обхода (п.2.7).

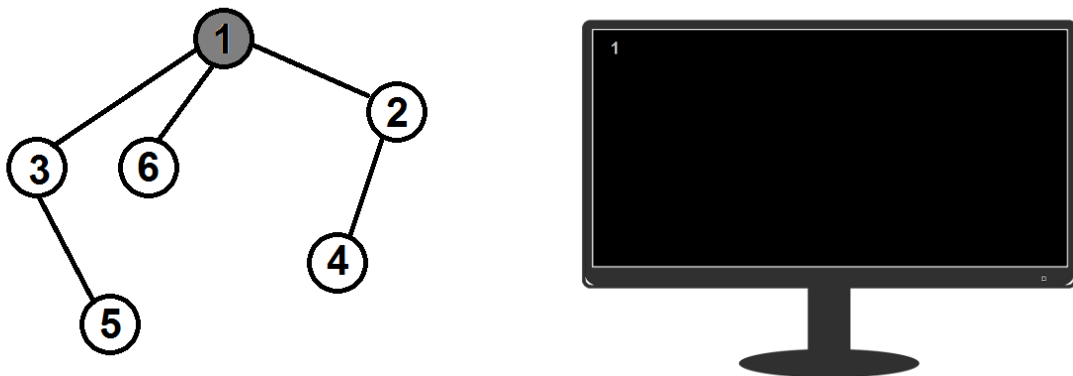
Например, пусть дан граф (рисунок 1), заданный в виде матрицы смежности:



$$G = \begin{Bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{Bmatrix}$$

Рисунок 1 – Граф

Тогда, если мы начнем обход из первой вершины, то на шаге 2.1 она будет помечена как посещенная ($NUM[1] = True$), на экран будет выведена единица.



$$NUM = \{True \quad False \quad False \quad False \quad False \quad False\}$$

Рисунок 2 – Вызов DFS(1)

При просмотре 1-й строки матрицы смежности

$$G = \begin{pmatrix} 0 & \boxed{1} & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

будет найдена смежная вершина с индексом 2 ($G(1,2) = 1$), которая не посещена ($NUM[2] = False$) и будет вызвана процедура обхода уже для нее - DFS(2).

На следующем вызове на шаге 2.1 вершина 2 будет помечена как посещенная ($NUM[2] = True$), на экран будет выведена двойка.

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.
- 3.* Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2*

1. Для матричной формы представления графов выполните преобразование рекурсивной реализации обхода графа к не рекурсивной.

Результат работы программы:

```
Введите количество вершин: 6

Матрица смежности:
 0  1  1  1  1  1
 1  0  0  1  1  0
 1  0  0  1  0  0
 1  1  1  0  1  1
 1  1  0  1  0  0
 1  0  0  1  0  0

Списки смежности:
Вершина 0: 1 -> 2 -> 3 -> 4 -> 5
Вершина 1: 0 -> 3 -> 4
Вершина 2: 0 -> 3
Вершина 3: 0 -> 1 -> 2 -> 4 -> 5
Вершина 4: 0 -> 1 -> 3
Вершина 5: 0 -> 3

Введите начальную вершину: 0

Обход в глубину (матрица смежности)
Путь обхода:  0  1  3  2  4  5

Обход в глубину (списки смежности)
Путь обхода:  0  1  3  2  4  5
```

Рисунок 1 – задание 1

```

Введите количество вершин: 6

Матрица смежности:
0 1 1 0 1 1
1 0 0 1 1 0
1 0 0 0 0 1
0 1 0 0 0 0
1 1 0 0 0 1
1 0 1 0 1 0

Списки смежности:
Вершина 0: 5 -> 4 -> 2 -> 1 -> NULL
Вершина 1: 4 -> 3 -> 0 -> NULL
Вершина 2: 5 -> 0 -> NULL
Вершина 3: 1 -> NULL
Вершина 4: 5 -> 1 -> 0 -> NULL
Вершина 5: 4 -> 2 -> 0 -> NULL

Введите начальную вершину: 0

=== Обход в глубину (матрица смежности) ===
Путь обхода: 0 1 3 2 4 5

=== Обход в глубину (списки смежности) ===
Путь обхода: 0 1 3 2 4 5

```

Рисунок 2 – задание 2

Вывод: в ходе выполнения лабораторной работы ознакомились с алгоритмом обхода графа в глубину и освоили его практическую реализацию.

Приложение

Листинг программы 1

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

struct Node {
    int vertex;
    struct Node* next;
};

void addEdgeSorted(struct Node** adjList, int src, int dest) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = dest;
    newNode->next = NULL;

    if (adjList[src] == NULL || dest < adjList[src]->vertex) {
        newNode->next = adjList[src];
        adjList[src] = newNode;
    }
    else {
        struct Node* current = adjList[src];

```

```

        while (current->next != NULL && current->next->vertex < dest)
        {
            current = current->next;
        }
        newNode->next = current->next;
        current->next = newNode;
    }

    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = src;
    newNode->next = NULL;

    if (adjList[dest] == NULL || src < adjList[dest]->vertex) {
        newNode->next = adjList[dest];
        adjList[dest] = newNode;
    }
    else {
        struct Node* current = adjList[dest];
        while (current->next != NULL && current->next->vertex < src) {
            current = current->next;
        }
        newNode->next = current->next;
        current->next = newNode;
    }
}

void DFS_matrix(int** G, int numG, int* visited, int s) {
    visited[s] = 1;
    printf("%3d", s);

    for (int i = 0; i < numG; i++) {
        if (G[s][i] == 1 && visited[i] == 0)
            DFS_matrix(G, numG, visited, i);
    }
}

void DFS_list(struct Node** adjList, int numG, int* visited, int s) {
    visited[s] = 1;
    printf("%3d", s);

    struct Node* temp = adjList[s];
    while (temp != NULL) {
        int adjacentVertex = temp->vertex;
        if (visited[adjacentVertex] == 0) {
            DFS_list(adjList, numG, visited, adjacentVertex);
        }
        temp = temp->next;
    }
}

void printAdjacencyList(struct Node** adjList, int numG) {
    printf("\nСписки смежности:\n");
    for (int i = 0; i < numG; i++) {
        printf("Вершина %d: ", i);
        struct Node* temp = adjList[i];
        while (temp != NULL) {
            printf("%d", temp->vertex);
            temp = temp->next;
        }
    }
}

```

```

        if (temp != NULL) {
            printf(" -> ");
        }
    }
    printf("\n");
}

}

void freeAdjacencyList(struct Node** adjList, int numG) {
    for (int i = 0; i < numG; i++) {
        struct Node* temp = adjList[i];
        while (temp != NULL) {
            struct Node* toDelete = temp;
            temp = temp->next;
            free(toDelete);
        }
    }
    free(adjList);
}

int main() {
    setlocale(LC_ALL, "Russian");

    int** G;
    int numG, current;
    int* visited;
    struct Node** adjList;

    printf("Введите количество вершин: ");
    scanf("%d", &numG);

    visited = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++)
        G[i] = (int*)malloc(numG * sizeof(int));

    adjList = (struct Node**)malloc(numG * sizeof(struct Node*));
    for (int i = 0; i < numG; i++) {
        adjList[i] = NULL;
    }

    srand(time(NULL));

    for (int i = 0; i < numG; i++) {
        visited[i] = 0;
        for (int j = i; j < numG; j++) {
            if (i == j) {
                G[i][j] = 0;
            }
            else {
                int value = rand() % 2;
                G[i][j] = value;
                G[j][i] = value;

                if (value == 1) {
                    addEdgeSorted(adjList, i, j);
                }
            }
        }
    }
}

```

```

    }
}

printf("\nМатрица смежности:\n");
for (int i = 0; i < numG; i++) {
    for (int j = 0; j < numG; j++) {
        printf("%3d", G[i][j]);
    }
    printf("\n");
}

printAdjacencyList(adjList, numG);

printf("\nВведите начальную вершину: ");
scanf("%d", &current);

printf("\n Обход в глубину (матрица смежности) \n");
for (int i = 0; i < numG; i++) visited[i] = 0;
printf("Путь обхода: ");
DFS_matrix(G, numG, visited, current);
printf("\n");

printf("\n Обход в глубину (списки смежности) \n");
for (int i = 0; i < numG; i++) visited[i] = 0;
printf("Путь обхода: ");
DFS_list(adjList, numG, visited, current);
printf("\n");

free(visited);
for (int i = 0; i < numG; i++)
    free(G[i]);
free(G);
freeAdjacencyList(adjList, numG);

printf("\nПрограмма завершена.\n");
return 0;
}

```

Листинг программы 2

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

struct Stack {
    int top;
    int capacity;
    int* array;
};

struct Node {
    int vertex;
    struct Node* next;
};

struct Stack* createStack(int capacity) {

```

```

    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
    stack->capacity = capacity;
    stack->top = -1;
    stack->array = (int*)malloc(capacity * sizeof(int));
    return stack;
}

int isStackEmpty(struct Stack* stack) {
    return stack->top == -1;
}

void push(struct Stack* stack, int item) {
    stack->array[++stack->top] = item;
}

int pop(struct Stack* stack) {
    if (isStackEmpty(stack))
        return -1;
    return stack->array[stack->top--];
}

void addEdge(struct Node** adjList, int src, int dest) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = dest;
    newNode->next = adjList[src];
    adjList[src] = newNode;
}

void DFS_adjacency(struct Node** adjList, int numG, int* visited, int
start) {
    struct Stack* stack = createStack(numG);

    push(stack, start);
    visited[start] = 1;

    while (!isStackEmpty(stack)) {
        int current = pop(stack);
        printf("%3d", current);

        struct Node* temp = adjList[current];
        while (temp != NULL) {
            int neighbor = temp->vertex;
            if (visited[neighbor] == 0) {
                visited[neighbor] = 1;
                push(stack, neighbor);
            }
            temp = temp->next;
        }
    }

    free(stack->array);
    free(stack);
}

struct Node** matrixToAdjacencyList(int** G, int numG) {
    struct Node** adjList = (struct Node**)malloc(numG * sizeof(struct
Node*));

```

```

    for (int i = 0; i < numG; i++) {
        adjList[i] = NULL;
        for (int j = 0; j < numG; j++) {
            if (G[i][j] == 1 && i != j) {
                addEdge(adjList, i, j);
            }
        }
    }

    return adjList;
}

// Функция для печати списков смежности
void printAdjacencyList(struct Node** adjList, int numG) {
    printf("\nСписки смежности:\n");
    for (int i = 0; i < numG; i++) {
        printf("Вершина %d: ", i);
        struct Node* temp = adjList[i];
        while (temp != NULL) {
            printf("%d -> ", temp->vertex);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}

void freeAdjacencyList(struct Node** adjList, int numG) {
    for (int i = 0; i < numG; i++) {
        struct Node* temp = adjList[i];
        while (temp != NULL) {
            struct Node* toDelete = temp;
            temp = temp->next;
            free(toDelete);
        }
    }
    free(adjList);
}

void DFS_matrix(int** G, int numG, int* visited, int start) {
    struct Stack* stack = createStack(numG);

    push(stack, start);
    visited[start] = 1;

    while (!isStackEmpty(stack)) {
        int current = pop(stack);
        printf("%3d", current);

        for (int i = numG - 1; i >= 0; i--) {
            if (G[current][i] == 1 && visited[i] == 0) {
                visited[i] = 1;
                push(stack, i);
            }
        }
    }

    free(stack->array);
    free(stack);
}

```

```

}

int main() {
    setlocale(LC_ALL, "Russian");

    int** G;
    int numG, current;
    int* visited;

    printf("Введите количество вершин: ");
    scanf("%d", &numG);

    visited = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++)
        G[i] = (int*)malloc(numG * sizeof(int));

    srand(time(NULL));

    for (int i = 0; i < numG; i++) {
        visited[i] = 0;
        for (int j = i; j < numG; j++) {
            if (i == j) {
                G[i][j] = 0;
            }
            else {
                int value = rand() % 2;
                G[i][j] = value;
                G[j][i] = value;
            }
        }
    }

    printf("\nМатрица смежности:\n");
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }

    struct Node** adjList = matrixToAdjacencyList(G, numG);
    printAdjacencyList(adjList, numG);

    printf("\nВведите начальную вершину: ");
    scanf("%d", &current);

    printf("\n=== Обход в глубину (матрица смежности) ===\n");
    for (int i = 0; i < numG; i++) visited[i] = 0;
    printf("Путь обхода: ");
    DFS_matrix(G, numG, visited, current);
    printf("\n");

    printf("\n=== Обход в глубину (списки смежности) ===\n");
    for (int i = 0; i < numG; i++) visited[i] = 0;
    printf("Путь обхода: ");
    DFS_adjacency(adjList, numG, visited, current);
    printf("\n");
}

```

```
    free(visited);  
    for (int i = 0; i < numG; i++)  
        free(G[i]);  
    free(G);  
    freeAdjacencyList(adjList, numG);  
  
    printf("\nПрограмма завершена.\n");  
    return 0;  
}
```